

Build Order

1. lib/socket.js ← connection setup
2. store/chat.js ← Zustand for chat state
3. app/(main)/chat/page.jsx ← conversation list
4. app/(main)/chat/[userId]/page.jsx ← chat window
5. components/chat/ConversationList.jsx
6. components/chat/ChatWindow.jsx
7. components/chat/MessageBubble.jsx
8. components/chat/MessageInput.jsx

Think Like an Engineer — Questions to ask before writing any file

Before lib/socket.js

What is this file's job?

- Create ONE socket instance shared across the whole app
- Why one? Multiple connections = multiple rooms = missed messages

What does the client need to do?

- Connect after login
- Disconnect on logout

What library?

- socket.io-client (install it first)

Before store/chat.js

What state does chat need?

- messages (array of current conversation)
- activeRoom (string – current roomId)
- onlineUsers (optional)

What actions does it need?

- setMessages
- appendMessage (new incoming message)
- setRoom
- clearChat (on conversation switch)

Why Zustand and not useState?

- Socket events happen OUTSIDE React components
- Zustand can be accessed without hooks

- `socket.on("receiveMessage")` needs to call `setMessages`
- can't use `useState` there

Before `chat/page.jsx`

What does this page show?

- list of people you've chatted with

Where does this data come from?

- GET `/api/messages` – but your backend might not have a "get all conversations" route yet
- Check your backend first before building UI

What happens on click?

- navigate to `/chat/[userId]`

Before `chat/[userId]/page.jsx`

What does this page do?

- Gets `userId` from `params`
- Connects socket on mount
- Emits `joinRoom` with `{ senderId: currentUser._id, receiverId: userId }`
- Listens for `oldMessages` → sets in store
- Listens for `receiveMessage` → appends to store
- Disconnects/leaves room on unmount

What is a "room"?

- `[userId1, userId2].sort().join("_")`
- Same two people always get same room regardless of who initiated
- Backend handles this – you just need to emit `joinRoom`

Before `MessageBubble.jsx`

What data does one message have?

- `content` (text)
- `sender._id`
- `createdAt`

How do you know if it's sent or received?

- `message.sender._id === currentUser._id`

→ sent = right side, blue/accent
→ received = left side, gray

The Real Engineer Mindset

Before writing any file ask:

1. What is this file's ONLY job?
2. What data does it need?
3. Where does that data come from?
4. What does it do with that data?
5. What does it give back or emit?

If you can answer all 5 → start writing.

If you can't → think more before touching keyboard.